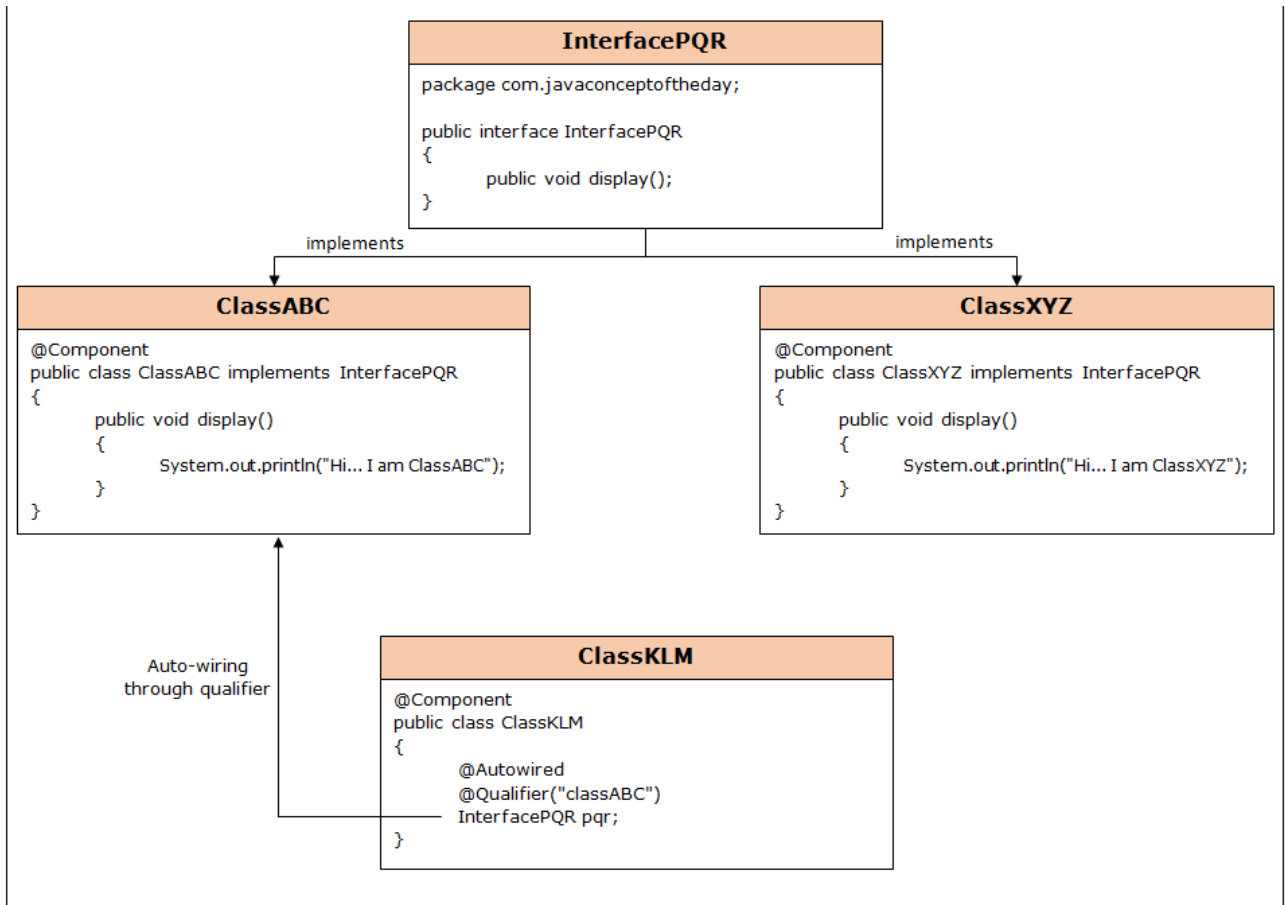


1. Spring boot Autowired and Qualifier



2. Springboot properties file

application.properties

app.example.integer-value=9876
app.example.decimal-value=1.75
app.example.boolean-value=true

the fields with will be set
to the values from the
application.properties
file

Java class

```
@Value("${app.example.integer-value}")  
private Integer valueInteger;  
  
@Value("${app.example.decimal-value}")  
private Double valueDouble;  
  
@Value("${app.example.boolean-value}")  
private Boolean valueBoolean;
```

3. Spring Boot @RequestParam::

Registration

localhost:8080/SpringMvcFormExample/register

Spring MVC Form Demo - Registration

User Name:

Password:

E-mail:

Birthday (mm/dd/yyyy):

Profession:

4. Boot @PathVariable and @RequestParam(QueryString)::

http://localhost:8080/users/101/accounts?type=current&status=active

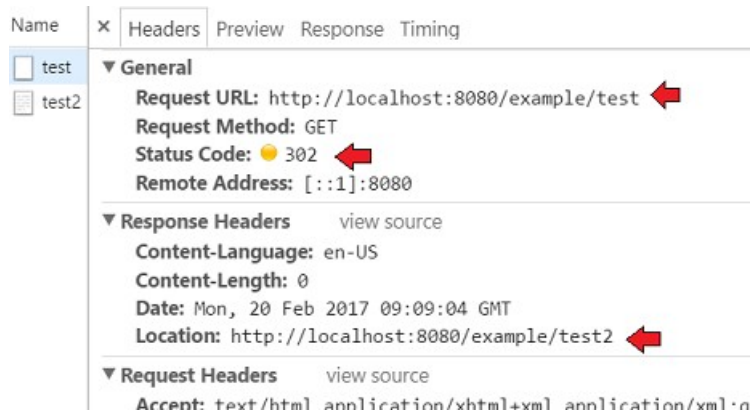
Path Variable

Request Parameter

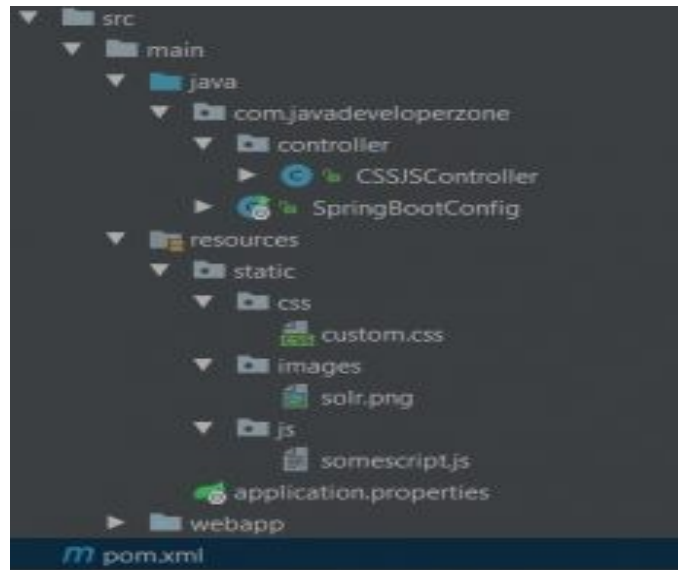
Request Parameter

```
@GetMapping("/user/{id}/accounts")  
public List<Account> getAccounts(@PathVariable String id, @RequestParam String type, @RequestParam String status) {  
    //...  
}
```

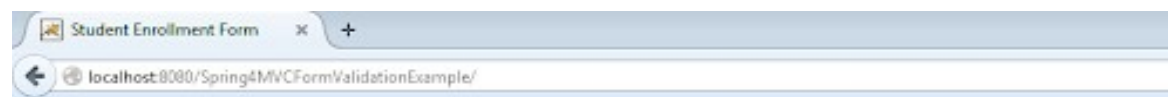
5.Spring Boot Redirection::



6. Spring boot css + javascript + images:::



7. Spring Boot Form validation + create User defined .properties file to store the error messages :::



Enrollment Form

First Name

First Name must be between 3 and 30 characters long

Last Name

Last Name must be between 3 and 30 characters long

Sex

☒ Male ☐ Female

Please specify your gender

Date of birth

Date of birth can not be blank

Email

Email can not be blank

Section

☒ Graduate ☐ Post Graduate ☐ Research

Please select your section

Country

Please select your country

First Attempt ?



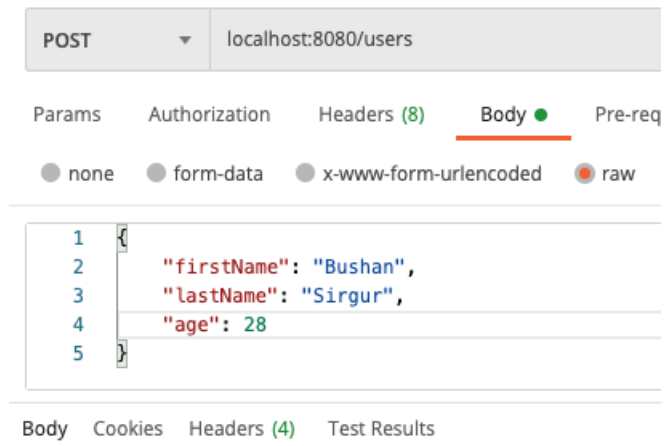
Subjects

☒ Chemistry
☐ Life Science
☐ Political Science

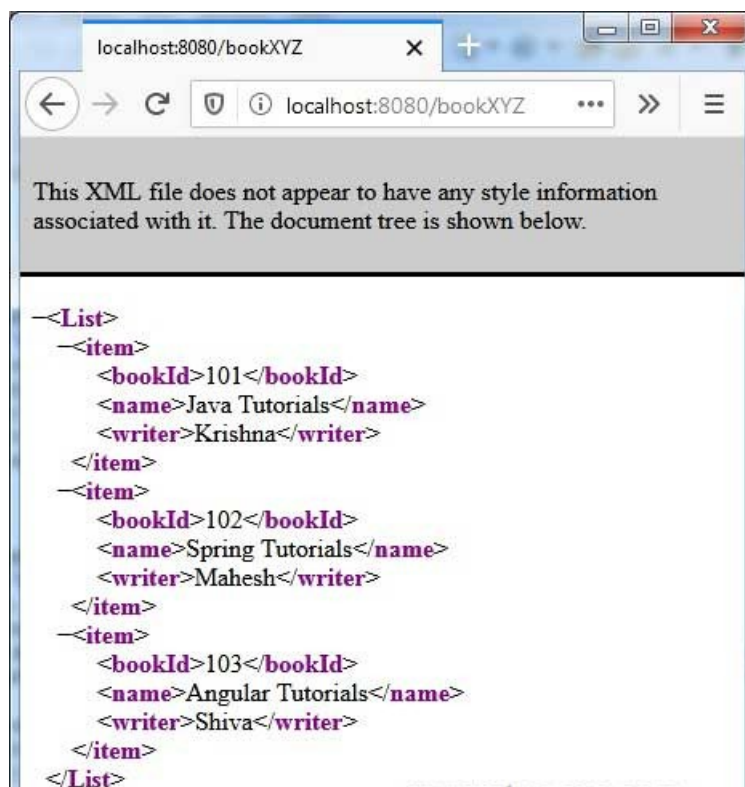
Please select at least one subject

Register

8.@RequestBody :::



9.@ResponseBody::::



10.Spring Boot Form validation + create User defined .properties file to store the error messages :::

Customer Save Page

localhost:9090/SpringFormValidation/cust/save.do

Apps Java Utility Apple Android Personal jQuery Spring Database

Name: *Customer name should be between 2 and 30 characters long*

Email: *Email is a required field*

Age: *Customer age should be in years*

Gender: *This is a required field*

Birthday: *This is a required field*

Phone: *Invalid format, valid formats are 1234567890, 123-456-7890 x1234*

11.@RestController::

localhost:8080/springrest/ x

localhost:8080/springrest/customers

Raw Parsed

```
[
  {
    "id": 101,
    "firstName": "John",
    "lastName": "Doe",
    "email": "djohn@gmail.com",
    "mobile": "121-232-3435",
    "dateOfBirth": 1467240235069
  },
  {
    "id": 201,
    "firstName": "Russ",
    "lastName": "Smith",
    "email": "sruss@gmail.com",
    "mobile": "343-545-2345",
    "dateOfBirth": 1467240235069
  },
  {
    "id": 301,
    "firstName": "Kate",
    "lastName": "Williams",
    "email": "kwilliams@gmail.com",
    "mobile": "876-237-2987",
    "dateOfBirth": 1467240235069
  }
]
```

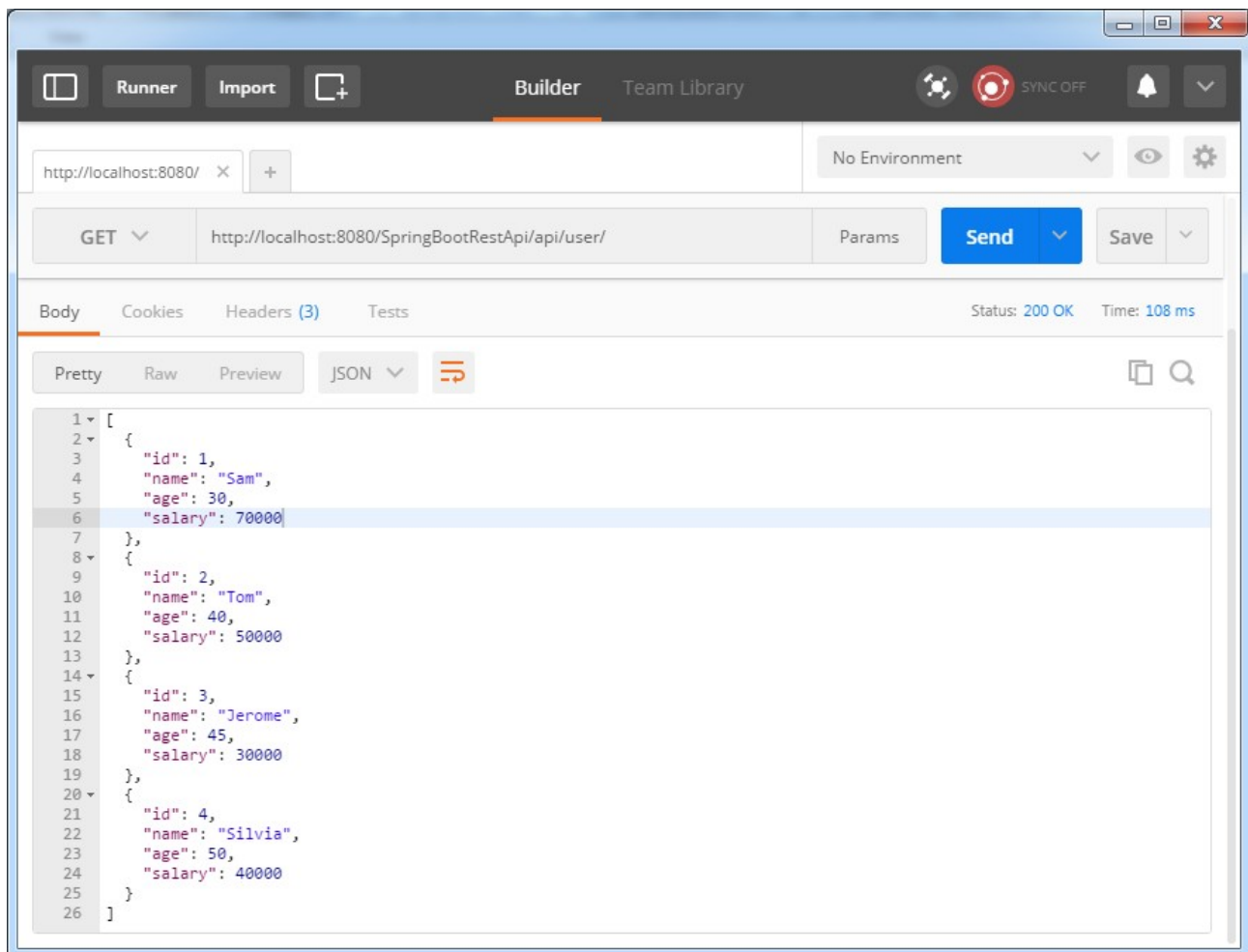
12.Spring Boot Rest Controller + JDBC (MYSQL):::

Hint: Create Mysql table with fields as bookid,bookname,author,price.

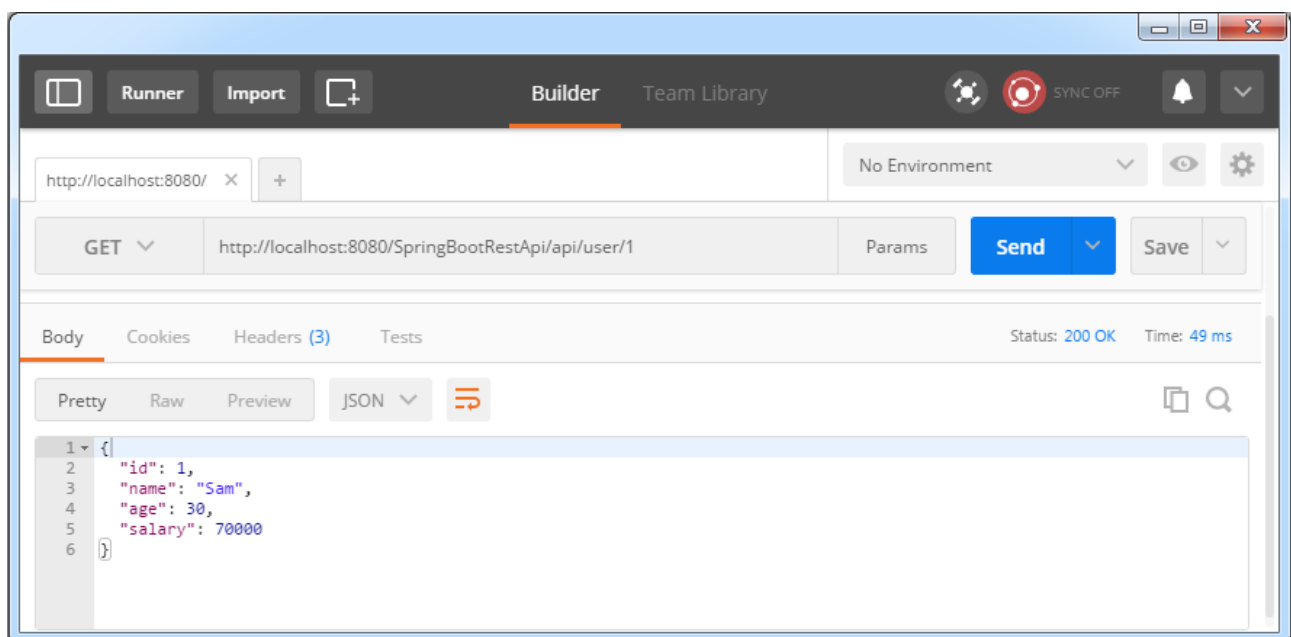
```
[
  {
    "bookid": 982,
    "bookname": "Programming with Java",
    "author": "E. Balagurusamy",
    "price": 350
  },
  {
    "bookid": 5433,
    "bookname": "Core and Advance Java",
    "author": "R. Nageswara Rao",
    "price": 800
  },
  {
    "bookid": 6321,
    "bookname": "Data Structures and Algorithms in Java",
    "author": "Robert Lafore",
    "price": 590
  },
  {
    "bookid": 6830,
    "bookname": "Exploring C ",
    "author": "Yashavant Kanetkar",
    "price": 670
  }
]
```

13.SPRING BOOT + HIBERNATE (MYSQL) + POSTMAN CRUD::

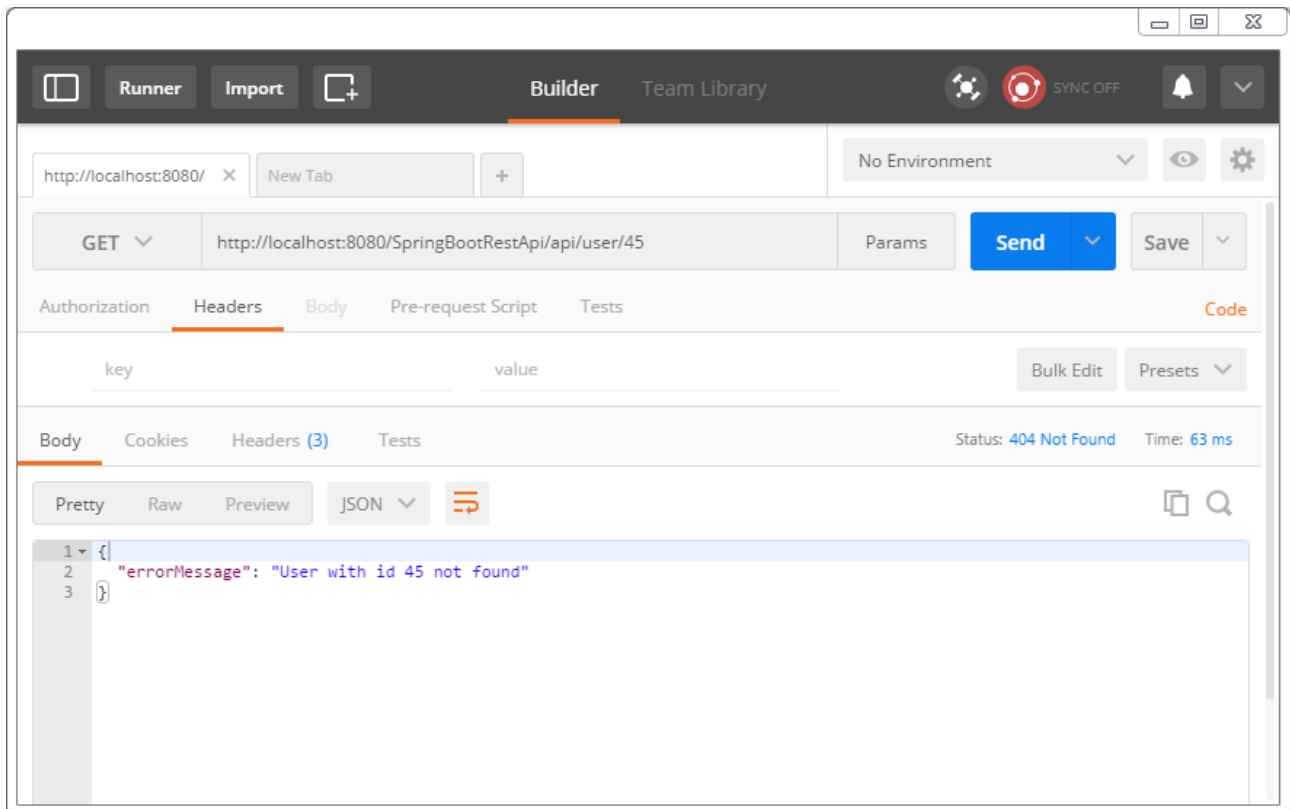
Retrieve all users:::



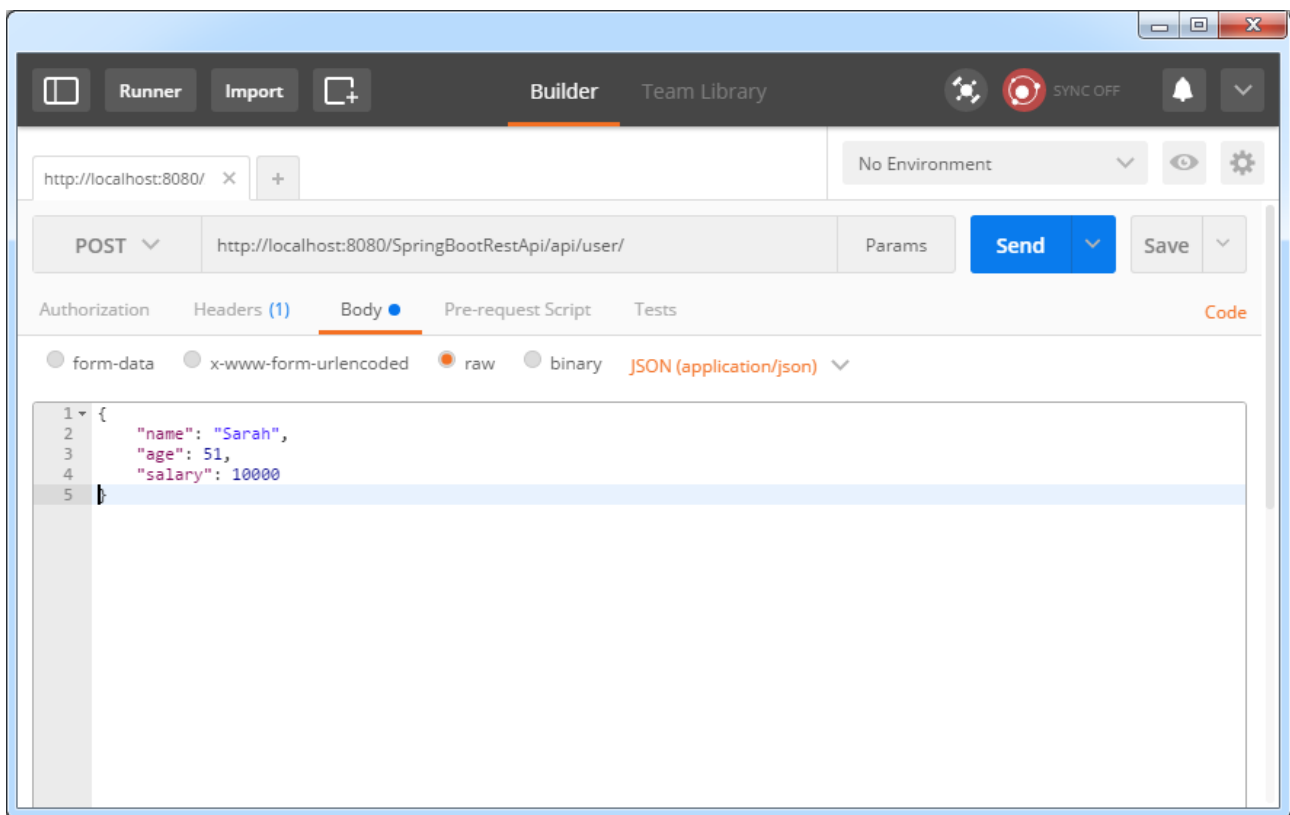
Retrieve a single user:::



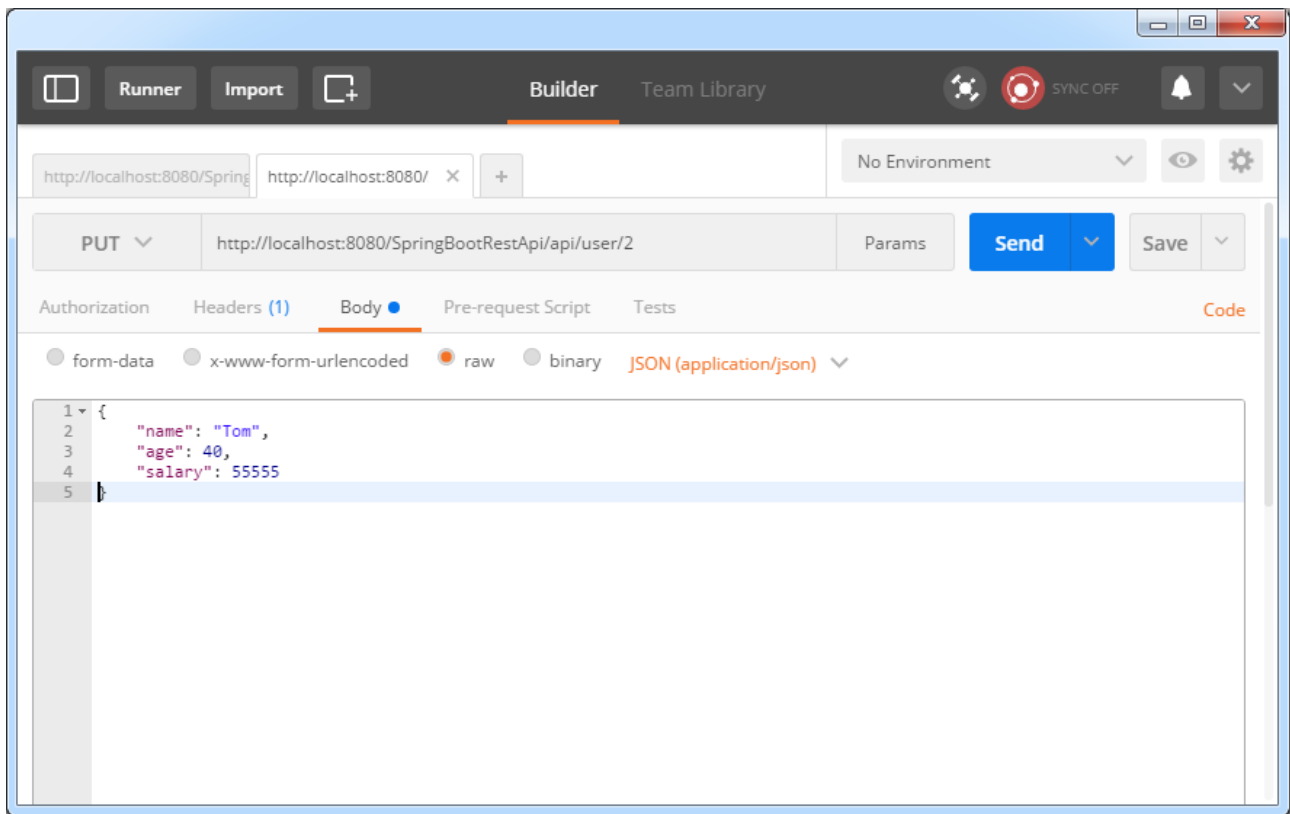
Retrieve an unknown user :::



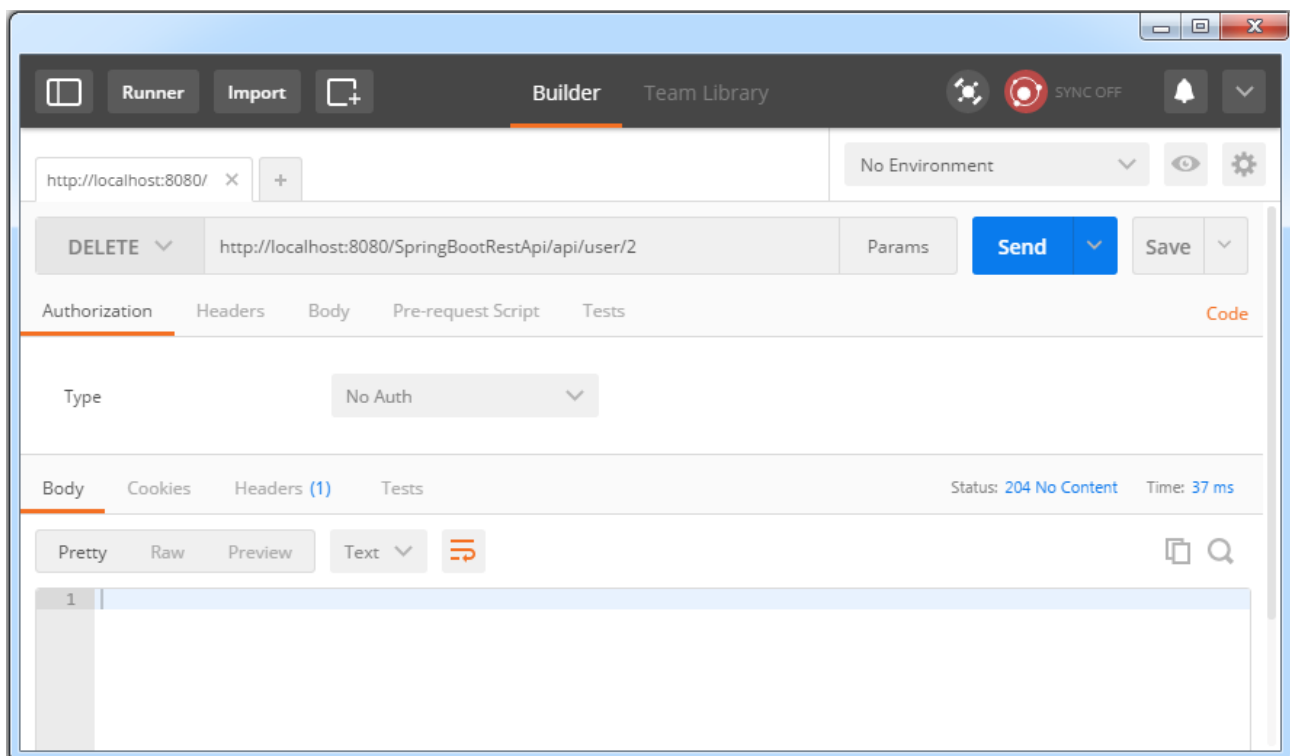
Create a User :::



Update an existing user ::



Delete an existing user ::



Build a Spring Boot RESTful web service using Spring Data JPA that demonstrates a **One-to-Many relationship** between two entities: `Restaurant` and `Food`. Use **MySQL** for persistence and **Postman** for testing the API endpoints. Apply proper validation and handle record not found scenarios gracefully.

🌟 Instructions:

1. Entities

- Create a `Restaurant` entity with fields:
 - `id` : Long (Primary key)
 - `name` : String (Not blank)
 - `location` : String (Not blank)
- Create a `Food` entity with fields:
 - `id` : Long (Primary key)
 - `name` : String (Not blank)
 - `price` : Double (Not null)
 - `restaurant` : Many-to-One relationship with `Restaurant`

2. Relationships

- One `Restaurant` can have many `Food` items.
- Map this using `@OneToMany` and `@ManyToOne` annotations.

3. Validation

- Use `@NotBlank` and `@NotNull` annotations to validate incoming request data.
- If validation fails, return a detailed error message with the list of invalid fields.

4. Exception Handling

- Handle `ResourceNotFoundException` for missing restaurant or food records.
- Use `@RestControllerAdvice` to handle exceptions globally and return proper error responses with:
 - HTTP status code
 - Error message
 - Timestamp

5. CRUD Operations via REST API

Implement the following endpoints:

Method	Endpoint	Description
POST	<code>/api/restaurants</code>	Create a new restaurant
GET	<code>/api/restaurants</code>	Get all restaurants
GET	<code>/api/restaurants/{id}</code>	Get restaurant by ID
DELETE	<code>/api/restaurants/{id}</code>	Delete restaurant by ID
POST	<code>/api/restaurants/{id}/food</code> 	Add a food item to a restaurant

DELETE

`/api/restaurants/foods/{foodId}`

Delete a food item by ID

6. Database

- Use MySQL database and configure connection in `application.properties`.
- Use JPA repositories (`JpaRepository`) for data access.

7. Testing

- Use **Postman** to test each endpoint.
- Include:
 - Valid request examples
 - Requests with validation errors
 - Requests for non-existent records

15. SPRING BOOT + HIBERNATE(MYSQL) + THYMELEAF TEMPLATE:

CRUD Operations

Thymeleaf



List of Clients

Add Client

ID	Name	Email	Phone	Address	Action
18	Bill Grant	bill.grant@gmail.com	1-254-365-3311	New York City, USA	Edit Delete
15	Lillian Lewis	lillian.lewis@gmail.com	1-441-498-4523	New York, USA	Edit Delete
14	Sebastian Grant	sebastian.grant@outlook.com	1-339-932-9323	Florida, USA	Edit Delete
13	Neil Wallace	neil.wallace@yahoo.com	1-585-897-0746	Jonesboro, USA	Edit Delete
12	Ava Wallace	ava.wallace@gmail.com	1-566-905-1715	Indianapolis, USA	Edit Delete
11	Frank Hodges	frank.hodges@outlook.com	1-785-387-9880	Rochester, USA	Edit Delete
10	Lara Lee	lara.lee@gmail.com	1-441-498-4553	New York, USA	Edit Delete
9	Maria Hernandez	maria.hernandez@gmail.com	1-339-932-9342	Frederick, USA	Edit Delete
8	Angela Knox	angela.knox@yahoo.com	1-585-897-0754	Grand Rapids, USA	Edit Delete
7	Harry Russell	harry.russell@gmail.com	1-566-905-1747	Florida, USA	Edit Delete

16. Build a Spring Boot application that **uses RestTemplate to consume a RESTful web service** (Spring Boot app or public API). Understand how to send GET, POST, PUT, and DELETE requests using RestTemplate, and how to handle responses and errors properly.

Hint: Consume a **public API** (like <https://jsonplaceholder.typicode.com>) using RestTemplate

17. To implement and test CRUD operations of a **Spring Boot REST API** for managing grocery items. You will write:

- **Unit tests** for service layer using Mockito
- **Controller tests** using MockMvc (@WebMvcTest)
- **Integration tests** using @SpringBootTest

Application Use Case

Create a Spring Boot application to manage **grocery items**.

Entity: GroceryItem

Field	Type
id	Long
name	String
quantity	int
price	double

PART A: Application Development

Required Components

- **Entity:** GroceryItem
- **Repository:** GroceryItemRepository
- **Service Layer:** GroceryItemService
- **Controller Layer:** GroceryItemController
- **Validation:** Add basic validations (@NotBlank, @Min, etc.)

PART B: Testing Requirements

1. Unit Testing – GroceryItemServiceTest.java

Test GroceryItemService methods using Mockito.

2. Controller Testing – GroceryItemControllerTest.java

Use @WebMvcTest with MockMvc to test controller endpoints in isolation.

3. Integration Testing – GroceryItemIntegrationTest.java

Test complete flow using @SpringBootTest and real components.